

3 Problem Statement

Building software prototypes from scratch is frustrating and time-consuming, especially for non-technical founders, design students, and indie developers who have ideas but struggle to translate them into structured requirements and working code. We noticed that existing tools either require technical expertise (like writing detailed specifications) or produce generic outputs that don't capture the specific context of what you're building.

When you're iterating on an idea, you often end up with scattered notes, lost requirements, and no clear way to track how your concept has evolved. There's no easy way to say *"build a meal-planning app"* and immediately get back user flows, acceptance criteria, and runnable code that actually matches your vision. We built this tool to bridge that gap — turning natural language descriptions into tangible, structured prototypes without requiring coding knowledge upfront.

4 Previous Work / Literature Survey

1. v0.dev by Vercel

Generates UI components from text prompts using AI — great for quickly getting React code, but heavily focused on frontend components rather than full product architecture. It doesn't handle workflow generation, requirement tracking, or domain-aware context merging across multiple prompts.

2. Figma AI (formerly Diagram)

Helps with design suggestions and component generation within Figma. While useful for visual design, it doesn't bridge the gap to actual code generation or structured requirement documentation. You're still working in the design silo without engineering outputs.

3. GitHub Copilot

Assists with code completion and generation based on comments and context. However, it's designed for developers who already know what they're building — it doesn't help non-technical users architect a product from a vague idea, nor does it generate workflows, acceptance criteria, or domain-specific layouts.

4. ChatGPT / Claude with custom prompts

While powerful, using general-purpose LLMs requires crafting detailed prompts and manual session management. There's no built-in domain detection, prompt merging logic, or persistent session tracking — you lose context when you start a new chat, and the output format varies each time.

We built **ProtoGen** — an AI-powered prototype generator with a React frontend and Node.js/Express backend that uses Google's Gemini API for intelligent generation. The system follows a multi-step pipeline:

→ **Feature Extraction**

`featureExtractor.js` analyzes the prompt using keyword matching against 10 predefined domains (food_delivery, ecommerce, social_media, etc.) before any LLM call.

→ **Domain Detection**

`domainDetector.js` uses Jaccard similarity to determine if a new prompt belongs to the same domain — if so, it merges them; if not, it triggers a domain change notification.

→ **LLM Generation**

`llmService.js` handles generation in two modes: "workflow" (architecture specs) and "workflow+code" (produces runnable HTML/JS files) via the Gemini API.

→ **Session Persistence**

MongoDB via Mongoose stores session data, prompt history, merged requirements, and a full change log across iterations.

The frontend uses React Context (*PrototypeContext.jsx*) for global state management, Tailwind CSS for styling, and a live code preview via *react-simple-code-editor*. Unlike existing tools, our solution maintains context across iterations, automatically detects domain switches, and outputs production-ready code with pre-defined design palettes.

Tool / Technology	Why We Used It
React 18	Component-based UI with hooks for state management
Vite	Fast dev server and optimised build tooling
Tailwind CSS	Utility-first styling without custom CSS files
React Router DOM	Client-side navigation between landing page and generator
Axios	HTTP client for frontend → backend API calls
Lucide React	Clean, modern icon library for the UI
React Simple Code Editor + PrismJS	Syntax-highlighted live code preview in the browser
Node.js + Express	Backend API server for generation requests and sessions
MongoDB + Mongoose	Persistent storage for sessions, history, requirements

Tool / Technology	Why We Used It
Google Gemini API (@google/genai)	LLM powering generation — chosen for large context window & JSON output
Compromise + Natural	NLP libraries for keyword extraction in the feature pipeline
CORS	Middleware enabling cross-origin requests (port 5173 ↔ 5001)
dotenv	Environment variable management for API keys

7

Impact to Society

Students & Hackathons

Instead of spending hours scaffolding a project, a CS student with a hackathon idea can describe their concept and get a working prototype in minutes.

Small Startups & Indie Devs

Reduces the barrier to testing ideas quickly without committing weeks to development or hiring a full team upfront.

Non-Technical Founders

Someone with no design background can still produce structured, well-thought-out prototypes with clear requirements — making it easier to pitch to investors or collaborators.

Education

Students learning software engineering can use this to see how natural language requirements translate into architecture and code — a real bridge between theory and practice.

While ProtoGen won't replace professional developers, it democratizes the early stages of product development — letting more people participate in building software. In a world where technical skills are often gatekeepers to innovation, tools like this give more people a seat at the table.